MLOps Assignment 2

Tracking a Pre-trained Sentiment Model with MLflow

Name: Divya K N
USN: 4GW23CI012
Subject: MLOps Assignment 2

Run Summary

Run	Batch size	Accuracy	Latency (s)
run_batch1	1	1.0	0.129
run_batch8	8	1.0	0.048
run_batch4	4	1.0	0.044

Model: distilbert-base-uncased-finetuned-sst-2-english. Dataset: 10 labelled sentences. Experiment: sentiment-inference-lab (MLflow 3.17.2). Values are from the terminal output of python mlflow_lab.py.

Task 3: Third Run

A third run, **run_batch4** (same model, batch_size=4), was added at the bottom of mlflow_lab.py. It ran successfully and appears alongside run_batch1 and run_batch8 in the MLflow UI.

Task 4: Short Report

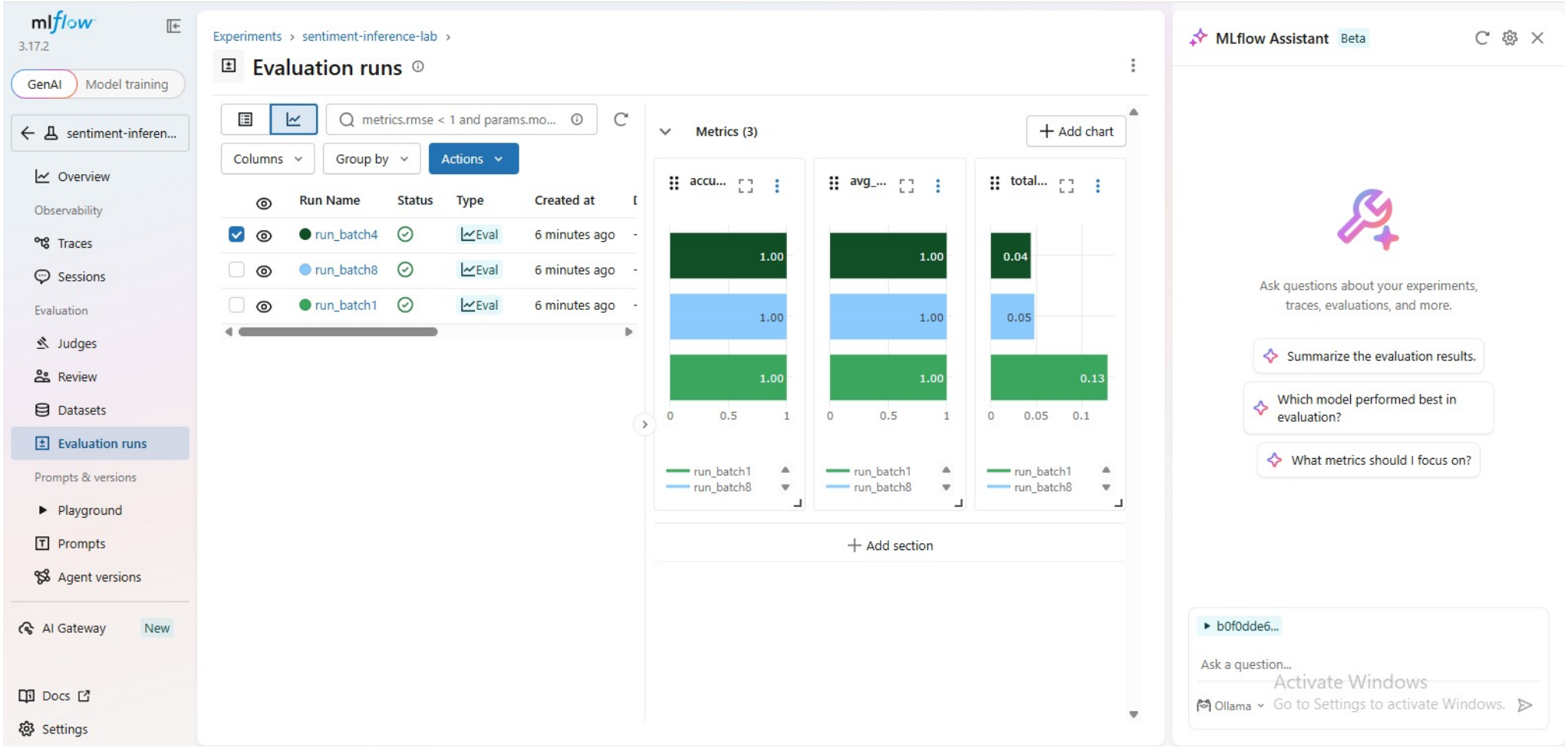
1. Which run had the lowest latency, and why might that be?
run_batch4 had the lowest latency (0.044 s), followed by run_batch8 (0.048 s) and run_batch1 (0.129 s). Processing several sentences together reduces repeated per-call inference overhead, so batch size 1 was clearly the slowest. The gap between batch 4 and batch 8 is very small and can change between executions because of warm-up and system conditions.

2. Did accuracy change between runs? Why or why not?
No. Accuracy stayed at 1.0 (100%) for all three runs. The same pretrained model and the same test sentences were used; changing the batch size only changes how inputs are grouped for inference, not the model weights or the predictions.

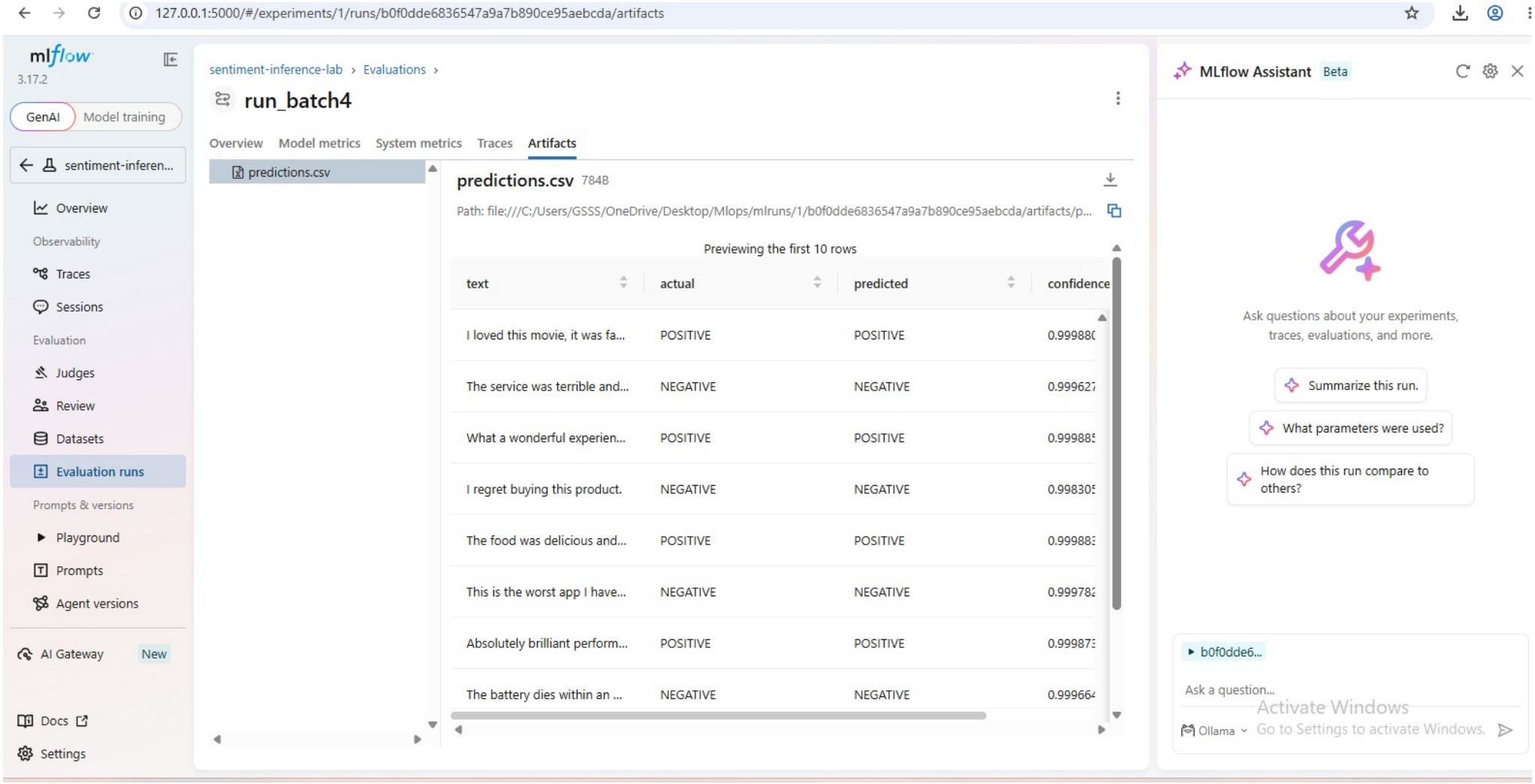
3. Name two benefits of using MLflow here instead of printing results in the terminal.
Reproducibility and record-keeping: MLflow stores parameters, metrics and artifacts such as predictions.csv for every run, so they can be inspected later.

Easy comparison: MLflow shows runs and their metrics side by side in charts, making accuracy, confidence and latency easier to compare than terminal output.

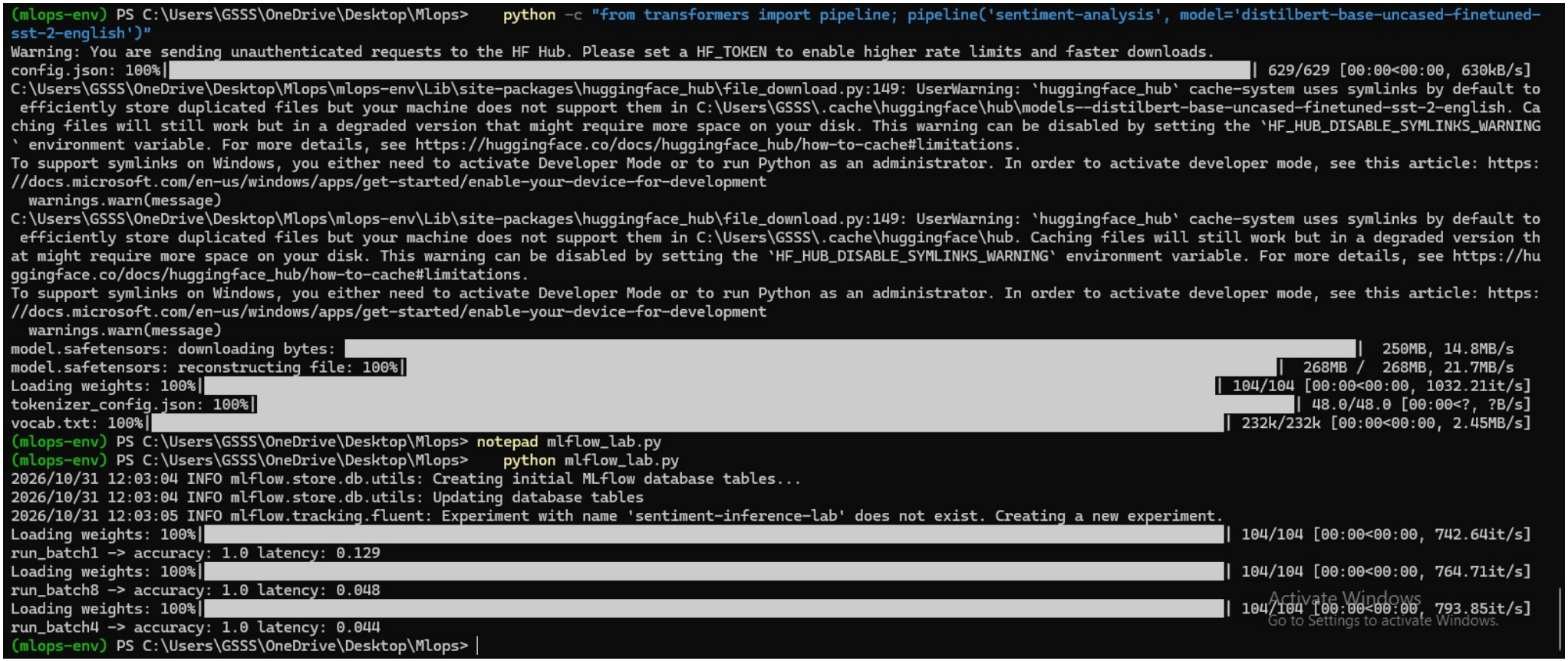
Screenshot of the MLflow runs table and comparison charts:



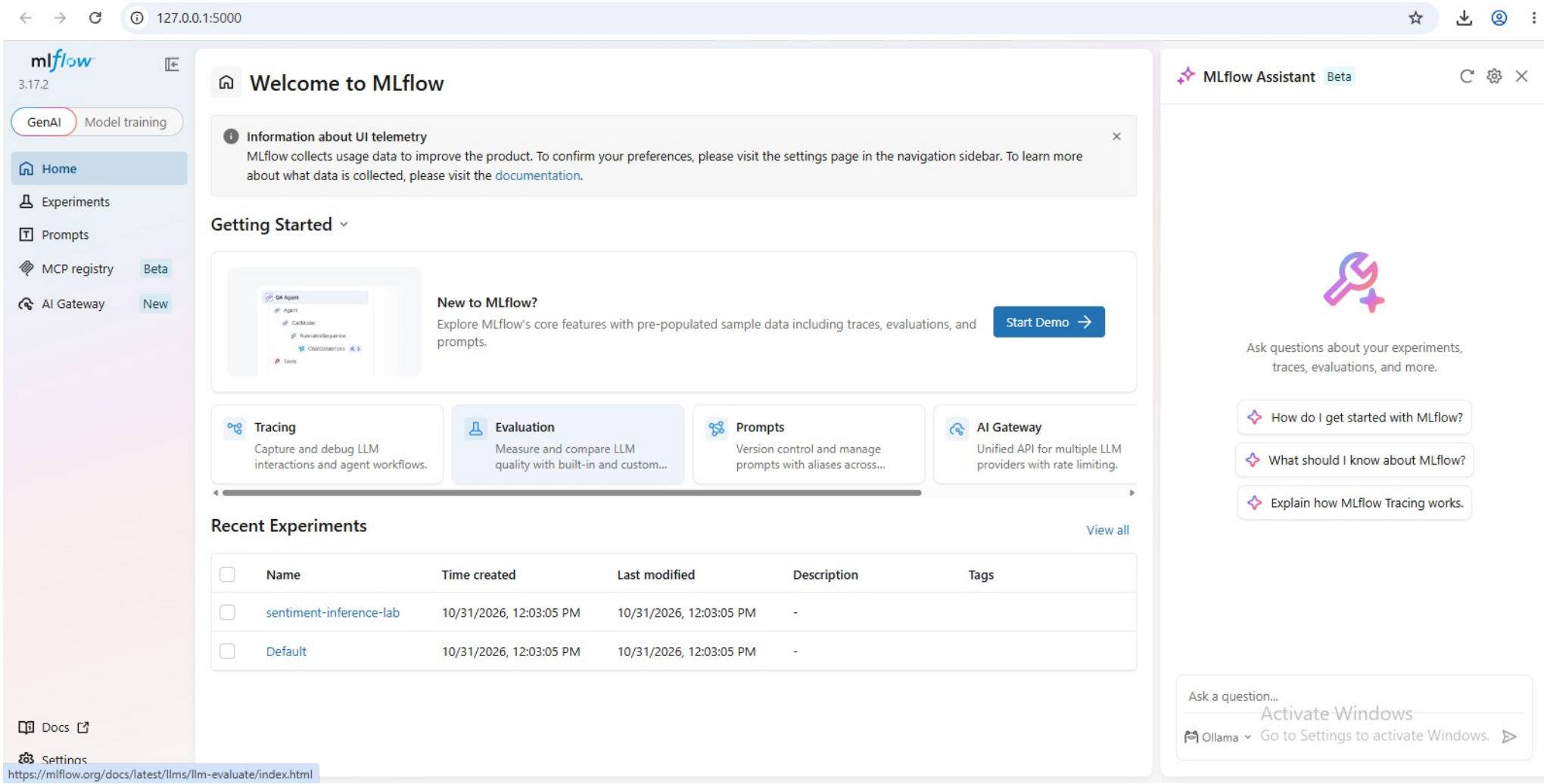
Screenshot of the run_batch4 artifact (predictions.csv):



Execution: Model download, script run and recorded metrics:



Execution: MLflow UI started, experiment sentiment-inference-lab created:



Script: mlflow_lab.py

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE"),
]
texts = [t for t, _ in data]
labels = [l for _, l in data]

mlflow.set_experiment("sentiment-inference-lab")


def run_experiment(model_name, batch_size, run_name):
    clf = pipeline("sentiment-analysis", model=model_name)

    with mlflow.start_run(run_name=run_name):
        # Parameters
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        # Inference
        start = time.time()
        preds = clf(texts, batch_size=batch_size)
        latency = time.time() - start

        pred_labels = [p["label"] for p in preds]
        accuracy = sum(p == l for p, l in zip(pred_labels, labels)) / len(labels)
        avg_conf = sum(p["score"] for p in preds) / len(preds)

        # Metrics
        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_conf)
        mlflow.log_metric("total_latency_sec", latency)

        # Artifact
        df = pd.DataFrame({
            "text": texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [p["score"] for p in preds],
        })
        df.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(run_name, "-> accuracy:", accuracy, "latency:", round(latency, 3))


MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")  # Task 3: third run
```